

Week 3 Exercises

Nick Hegge

07/17/2024

Please complete all exercises below. You may use any library that we have covered in class UP TO THIS POINT.

For each function, show that it works, by using the provided data as a test and by feeding in some test data that you create to test your function

Add comments to your function to explain what each line is doing

1.) Write a function that takes in a string with a person's name in the form

"Sheets, Dave"

and returns a string of the form

"Dave Sheets"

Note:

-assume no middle initial ever -remove the comma -be sure there is white space between the first and last name

You will probably want to use stringr

```
library(stringr)

reorder_name <- function(last_first) {

  last_first <- str_trim(last_first)

  name_parts <- str_split(last_first, ", ")[[1]]

  first_last <- paste(name_parts[2], name_parts[1], sep = " ")

  return(first_last)
}

# Inputting Name
name_in <- "Sheets, Dave"
reorder_name(name_in)
```

```
## [1] "Dave Sheets"
```

```
name_in <- "Hegge, Nick"
reorder_name(name_in)
```

```
## [1] "Nick Hegge"
```

2.) Write a function that takes in a string of values x, and returns a data frame with three columns, x, x² and the square root of x

```
powers_df <- function(x) {  
  df <- data.frame(x = x, x_squared = x^2, sqrt_x = sqrt(x))  
  return(df)  
}  
  
# Test Data  
x <- c(1, 3, 5, 7, 9, 11, 13)  
powers_df(x)
```

```
##      x x_squared  sqrt_x  
## 1  1         1 1.000000  
## 2  3         9 1.732051  
## 3  5        25 2.236068  
## 4  7        49 2.645751  
## 5  9        81 3.000000  
## 6 11       121 3.316625  
## 7 13       169 3.605551
```

3) Two Sum - Write a function named two_sum()

Given a vector of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9 Output: [0,1] Explanation: Because nums[0] + nums[1] == 9, we return [0, 1]. Example 2:

Input: nums = [3,2,4], target = 6 Output: [1,2] Example 3:

Input: nums = [3,3], target = 6 Output: [0,1]

Constraints:

2 <= nums.length <= 10⁴ -10⁹ <= nums[i] <= 10⁹ -10⁹ <= target <= 10⁹ Only one valid answer exists.

Note: For the first problem I want you to use a brute force approach (loop inside a loop)

The brute force approach is simple. Loop through each element x and find if there is another value that equals to target - x

Use the function seq_along to iterate

```
two_sum <- function(nums, target) {  
  n <- length(nums)  
  result <- list()  
  
  for (i in seq_along(nums)) {
```

```

    for (j in seq_along(nums)) {
      if (i != j && nums[i] + nums[j] == target) {
        pair <- sort(c(i, j))
        result <- c(result, list(pair))
      }
    }
  }
}

return(result)
}

# Test data

nums_vector <- c(5, 7, 12, 34, 6, 10, 8, 9)
target <- 13

z <- two_sum(nums_vector, target)
for (pair in z) {
  cat("[1]", pair[1], pair[2], "\n")
}

```

```

## [1] 1 7
## [1] 2 5
## [1] 2 5
## [1] 1 7

```

Okay tough problem coming here! Look carefully at Jeremiah's examples of using a hash to simplify a loop by using a hash.

4) Now write the same function using hash tables.

Loop the array once to make a hash map of the value to its index. Then loop again to find if the value of target-current value is in the map.

The keys of your hash table should be each of the numbers in the `nums_vector` minus the target.

A simple implementation uses two iterations. In the first iteration, we add each element's value as a key and its index (array position) as a value to the hash table. Then, in the second iteration, we check if each element's complement ($\text{target} - \text{nums_vector}[i]$) exists in the hash table. If it does exist, we return current element's index and its complement's index. Beware that the complement must not be `nums_vector[i]` itself!

```
require(hash)
```

```
## Loading required package: hash
```

```
## hash-2.2.6.3 provided by Decision Patterns
```

```

two_sum2 <- function(nums_vector, target) {
  hash_table <- hash()

  for (i in seq_along(nums_vector)) {

```

```

complement <- target - nums_vector[i]

if (!is.null(hash_table[[as.character(complement)]])) {
  result <- c(hash_table[[as.character(complement)]], i)
  if (result[1] < result[2]) {
    cat("# [1]", result[1] + 1, result[2] + 1, "\n")
    cat("# [1]", result[2] + 1, result[1] + 1, "\n")
  }
}

if (is.null(hash_table[[as.character(nums_vector[i])]])) {
  hash_table[[as.character(nums_vector[i])]] <- i
}
}
}

# Test data
nums_vector <- c(5, 7, 12, 34, 6, 10, 8, 9)
target <- 15

two_sum2(nums_vector, target)

```

```

## # [1] 2 7
## # [1] 7 2
## # [1] 3 8
## # [1] 8 3
## # [1] 6 9
## # [1] 9 6

```